

PID 제어 실습

Gazebo Classic 로봇 팔 Ziegler-Nichols PID 튜닝

cchyun@gmail.com

Part 1. 개요 및 원리

- **제어 실습 대상:** ROS2 Humble 및 Gazebo Classic 환경
- **실습 제어 목표:** 1자유도(1-DOF) 로봇 팔의 수평 자세(0.0 rad) 정밀 제어 및 유지
- **핵심 튜닝 방법:** 페루프 비례(P) 제어 상태에서 비례 이득(K_p)을 점진적으로 증가시켜, 시스템이 일정한 진폭으로 진동(등진폭 지속 진동)하는 안정 한계 상태를 탐색함

기호	의미	계측 및 도출 방법
K_u	임계 이득 (Ultimate Gain)	감쇄하지 않는 지속적인 등진폭 진동이 처음 발생하는 시점의 비례 게인 (K_p)
T_u	임계 주기 (Ultimate Period)	등진폭 진동 파형에서 피크와 피크 사이의 시간 간격 (단위: 초)

$$K_p = 0.6 K_u \quad K_i = 1.2 \frac{K_u}{T_u} \quad K_d = 0.075 K_u T_u$$

- 하드웨어 안전 주의 사항

- 임계 진동 상태는 제어 시스템이 불안정 발산하기 직전의 매우 위험한 상태임
- 실제 로봇 하드웨어에서 직접 조율을 시도하면 기어박스 파손이나 고속 충돌 사고를 초래할 수 있음
- 반드시 Gazebo 등 물리 시뮬레이션 환경에서 선검증 및 안전 한계를 도출한 뒤 적용해야 함

Part 2. 사전 준비 및 로봇 모델

- Gazebo Classic과 ROS2 연동 및 컨트롤러 구동을 위한 필수 라이브러리 설치

```
# apt 패키지 인덱스 업데이트 및 ros2_control 구동 패키지 설치
sudo apt update
sudo apt install -y ros-humble-gazebo-ros2-control \
                    ros-humble-joint-state-broadcaster \
                    ros-humble-effort-controllers
```

- URDF 모델의 링크 및 조인트 계층 구조

- `world`: 시뮬레이션 절대 원점 역할을 하는 고정 링크
- `base_link`: 지면에 고정된 원통형 로봇 받침대 (질량: 5.0 kg)
- `arm_link`: Y축 기준으로 회전하는 박스 형태의 1m 링크 (질량: 1.0 kg, 규격: $0.05 \times 0.05 \times 1.0$ m)
- `payload_link`: 로봇 팔 끝단에 부착된 구형 부하 질량 (기본 1.0 kg, Xacro 매개변수로 질량 가변 설정 가능)
- `arm_joint`: `base_link`와 `arm_link`를 연결하는 Y축 회전 연속 관절 (Continuous Joint)

로봇 팔 URDF 작성 (one_dof_arm.xacro)

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="one_dof_arm">

  <!-- 하중 설정을 위한 Xacro 인자 정의 -->
  <xacro:arg name="payload_mass" default="1.0"/>

  <!-- ===== WORLD LINK ===== -->
  <link name="world"/>

  <!-- ===== WORLD TO BASE JOINT ===== -->
  <joint name="world_to_base" type="fixed">
    <parent link="world"/>
    <child link="base_link"/>
    <origin xyz="0 0 0" rpy="0 0 3.14159"/>
  </joint>

  <!-- ===== BASE LINK ===== -->
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0.1" rpy="1.5708 0 0"/>
      <geometry>
        <cylinder radius="0.1" length="0.2"/>
      </geometry>
      <material name="grey">
        <color rgba="0.5 0.5 0.5 1"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0.1" rpy="1.5708 0 0"/>
      <geometry>
        <cylinder radius="0.1" length="0.2"/>
      </geometry>
    </collision>
    <inertial>
      <origin xyz="0 0 0.1" rpy="1.5708 0 0"/>
      <mass value="5.0"/>
      <inertia ixx="0.02916" iyy="0.02916" izz="0.025" ixy="0" ixz="0" iyz="0"/>
    </inertial>
  </link>
```


로봇 팔 URDF 작성 (one_dof_arm.xacro)

```
<!-- ===== ARM JOINT (토크 입력용 Effort 관절) ===== -->
<joint name="arm_joint" type="continuous">
  <parent link="base_link"/>
  <child link="arm_link"/>
  <origin xyz="0 0.125 0.1" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit effort="100.0" velocity="10.0"/>
  <dynamics damping="0.1" friction="0.1"/>
</joint>

<!-- ===== ARM LINK ===== -->
<link name="arm_link">
  <visual>
    <origin xyz="0.5 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="1.0 0.05 0.05"/>
    </geometry>
    <material name="blue">
      <color rgba="0.2 0.4 0.8 1"/>
    </material>
  </visual>
  <collision>
    <origin xyz="0.5 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="1.0 0.05 0.05"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0.5 0 0" rpy="0 0 0"/>
    <mass value="1.0"/>
    <inertia ixx="0.00042" iyy="0.08354" izz="0.08354" ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
```

로봇 팔 URDF 작성 (one_dof_arm.xacro)

```
<!-- ===== PAYLOAD JOINT ===== -->
<joint name="payload_joint" type="fixed">
  <parent link="arm_link"/>
  <child link="payload_link"/>
  <origin xyz="1.0 0 0" rpy="0 0 0"/>
</joint>

<!-- ===== PAYLOAD LINK (하중 가변 링크) ===== -->
<link name="payload_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
    <material name="red">
      <color rgba="0.8 0.2 0.2 1"/>
    </material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.05"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="$(arg payload_mass)"/>
    <inertia ixx="0.001" iyy="0.001" izz="0.001" ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
```

로봇 팔 URDF 작성 (one_dof_arm.xacro)

```
<!-- ===== ROS2 Control 하드웨어 인터페이스 설정 ===== -->
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="arm_joint">
    <command_interface name="effort"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>
</ros2_control>

<!-- ===== Gazebo 시스템 제어 플러그인 로드 ===== -->
<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <parameters>$(find my_robot_description)/config/arm_controllers.yaml</parameters>
    <robot_param>robot_description</robot_param>
    <robot_param_node>robot_state_publisher</robot_param_node>
    <hold_joints>>false</hold_joints>
  </plugin>
</gazebo>

<!-- ===== Gazebo 물리엔진용 색상 리소스 매핑 ===== -->
<gazebo reference="base_link">
  <material>Gazebo/Grey</material>
</gazebo>
<gazebo reference="arm_link">
  <material>Gazebo/Blue</material>
</gazebo>
<gazebo reference="payload_link">
  <material>Gazebo/Red</material>
</gazebo>

</robot>
```

Part 3. ros2_control 및 컨트롤러 설정

컨트롤러 설정 (arm_controllers.yaml)

- ros2_control 매니저 및 하위 조인트 컨트롤러 매개변수 구성

```
controller_manager:
  ros__parameters:
    update_rate: 1000 # 컨트롤러 제어 루프 주기 (1000Hz)

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    effort_controller:
      type: effort_controllers/JointGroupEffortController

effort_controller:
  ros__parameters:
    joints:
      - arm_joint
    command_interfaces:
      - effort
    state_interfaces:
      - position
      - velocity
```

- 컨트롤러 플러그인 상세 정의

- `joint_state_broadcaster`: 관절 상태 계측값을 수신하여 `/joint_states` 토픽으로 전역 발행함 (실시간 각도 모니터링용)
- `effort_controller`: 외부 제어 명령 토픽(`/effort_controller/commands`)을 구독하여 조인트에 실제 회전 토크를 실시간으로 인가함

Part 4. Launch 파일 및 PID 제어 노드

Launch 파일 구조 (pid_arm_launch.py)

- 로봇 설명 로드 및 가변 하중 인자 정의

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess, RegisterEventHandler, DeclareLaunchArgument
from launch.event_handlers import OnProcessExit
from launch.substitutions import Command, LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_share, 'urdf', 'one_dof_arm.xacro')

    # 끝단 페이로드(하중) 질량을 가변 인자로 구성
    declare_payload = DeclareLaunchArgument(
        'payload_mass',
        default_value='1.0',
        description='Payload mass in kg (default: 1.0kg)'
    )

    # Xacro 파싱 및 인자 주입
    robot_description = Command([
        'xacro ', xacro_file,
        ' payload_mass:=', LaunchConfiguration('payload_mass')
    ])

    ])
```


- 시뮬레이터 가동 및 로봇 모델 스폰 노드 구성

```
# Gazebo Classic 백엔드 구동 프로세스 정의
gazebo = ExecuteProcess(
    cmd=[
        'gazebo', '--verbose',
        '-s', 'libgazebo_ros_init.so',
        '-s', 'libgazebo_ros_factory.so',
    ],
    output='screen'
)

# 로봇 상태 출판 노드
rsp = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[{
        'robot_description': robot_description,
        'use_sim_time': True,
    }]
)

# Gazebo 월드에 URDF 로봇 팔 생성
spawn = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=[
        '-topic', 'robot_description',
        '-entity', 'one_dof_arm',
        '-z', '1.0',
    ],
    output='screen',
)
```

Launch 파일 구조 (pid_arm_launch.py)

- 컨트롤러 순차 구동 및 최종 런치 리스트 구성

```
# 로봇 스폰 완료 후 조인트 브로드캐스터 컨트롤러 가동
load_jsb = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['joint_state_broadcaster', '-c', '/controller_manager'],
    output='screen',
)

# 브로드캐스터 구동 완료 후 토크 제어기 가동
load_effort = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['effort_controller', '-c', '/controller_manager'],
    output='screen',
)

return LaunchDescription([
    declare_payload,
    gazebo,
    rsp,
    spawn,
    # 이벤트 핸들러를 통한 컨트롤러 순차적 기동 유도 (충돌 방지)
    RegisterEventHandler(
        event_handler=OnProcessExit(
            target_action=spawn,
            on_exit=[load_jsb],
        )
    ),
    RegisterEventHandler(
        event_handler=OnProcessExit(
            target_action=load_jsb,
            on_exit=[load_effort],
        )
    ),
])
```

- 클래스 정의 및 런타임 제어용 ROS2 파라미터 선언

```
import rclpy
import math
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Float64MultiArray
from rcl_interfaces.msg import SetParametersResult

class PIDArmController(Node):
    def __init__(self):
        super().__init__('pid_arm_controller')

        # --- 실시간 조율용 ROS2 파라미터 선언 ---
        self.declare_parameter('kp', 0.0)
        self.declare_parameter('ki', 0.0)
        self.declare_parameter('kd', 0.0)
        self.declare_parameter('setpoint', 0.0) # 목표 각도 (기본값: 수평 0.0 rad)
        self.declare_parameter('dt', 0.01) # 제어 주기 (기본값: 100Hz)
        self.declare_parameter('max_effort', 50.0) # 최대 인가 제한 토크 (보호 제한치)
        self.declare_parameter('gravity_gain', 0.0) # 중력 보상 이득
```

PID 제어 노드 작성 (pid_arm_control.py)

- 내부 멤버 변수 및 통신 엔드포인트 바인딩

```
self.kp = self.get_parameter('kp').value
self.ki = self.get_parameter('ki').value
self.kd = self.get_parameter('kd').value
self.setpoint = self.get_parameter('setpoint').value
self.dt = self.get_parameter('dt').value
self.max_effort = self.get_parameter('max_effort').value
self.gravity_gain = self.get_parameter('gravity_gain').value

# --- PID 내부 상태 저장 변수 ---
self.integral = 0.0
self.prev_error = 0.0
self.current_position = 0.0

# --- 퍼블리셔 및 서브스크라이버 바인딩 ---
self.joint_sub = self.create_subscription(
    JointState, '/joint_states', self.joint_callback, 10)
self.effort_pub = self.create_publisher(
    Float64MultiArray, '/effort_controller/commands', 10)

# --- 제어 루프 타이머 트리거 ---
self.timer = self.create_timer(self.dt, self.control_loop)

# --- 파라미터 실시간 변경 콜백 바인딩 ---
self.add_on_set_parameters_callback(self.param_callback)

self.get_logger().info(
    f'PID Arm Controller started | Kp={self.kp} Ki={self.ki} Kd={self.kd} | '
    f'Setpoint={self.setpoint} rad | gravity_gain={self.gravity_gain}'
)
```

PID 제어 노드 작성 (pid_arm_control.py)

- 파라미터 변경 콜백 및 조인트 상태 피드백 핸들러

```
def param_callback(self, params):
    for param in params:
        if param.name == 'kp':
            self.kp = param.value
        elif param.name == 'ki':
            self.ki = param.value
        elif param.name == 'kd':
            self.kd = param.value
        elif param.name == 'setpoint':
            self.setpoint = param.value
        elif param.name == 'gravity_gain':
            self.gravity_gain = param.value
    self.get_logger().info(
        f'Parameter updated | Kp={self.kp} Ki={self.ki} Kd={self.kd} | '
        f'Setpoint={self.setpoint} | GravityGain={self.gravity_gain}',
        throttle_duration_sec=1.0)
    return SetParametersResult(successful=True)

def joint_callback(self, msg: JointState):
    if 'arm_joint' in msg.name:
        idx = msg.name.index('arm_joint')
        # 센서로 측정한 현재 로봇의 각도 피드백을 실시간 업데이트
        self.current_position = msg.position[idx]
```

PID 제어 노드 작성 (pid_arm_control.py)

- 핵심 PID 및 Anti-Windup 수식 연산 제어 루프

```
def control_loop(self):
    # 오차(Error) 연산
    error = self.setpoint - self.current_position

    # Proportional (P 항)
    p_term = self.kp * error

    # Integral (I 항)
    self.integral += error * self.dt
    i_term = self.ki * self.integral

    # Derivative (D 항)
    derivative = (error - self.prev_error) / self.dt
    d_term = self.kd * derivative
    self.prev_error = error

    # 피드포워드 중력 보상 토크 계산
    ref_angle = self.setpoint
    gravity_comp = - self.gravity_gain * math.cos(ref_angle)

    # 토크 합산
    effort = p_term + i_term + d_term + gravity_comp

    # Anti-windup (Clamping 구조의 과누적 방지 처리)
    if effort > self.max_effort:
        self.integral -= error * self.dt # 출력이 상한 포화 시 적분 누적 일시 차단
        effort = self.max_effort
    elif effort < -self.max_effort:
        self.integral -= error * self.dt # 출력이 하한 포화 시 적분 누적 일시 차단
        effort = -self.max_effort

    # 토크 지령 퍼블리시
    msg = Float64MultiArray()
    msg.data = [effort]
    self.effort_pub.publish(msg)

    # 모니터링 로그 출력 (0.5초 주기로 스톱틀링)
    self.get_logger().info(
        f'pos={self.current_position:+.4f} | err={error:+.4f} | '
        f'eff={effort:+.3f} | P={p_term:+.2f} I={i_term:+.2f} D={d_term:+.2f} G={gravity_comp:+.2f}',
        throttle_duration_sec=0.5
    )
```

- 메인 진입점 및 노드 수명 관리

```
def main(args=None):
    rclpy.init(args=args)
    node = PIDArmController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # 노드 종료 및 가동 중단 시 로봇 팔 추락 방지 및 급제동을 위해 0 토크 인가 (안전 설계)
        zero = Float64MultiArray()
        zero.data = [0.0]
        node.effort_pub.publish(zero)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

- 동적 실시간 튜닝 지원: `ros2 param set` 명령어를 활용하여 노드 재시작 없이 즉시 파라미터 조율 가능

Part 5. Gazebo 실행 및 Ziegler-Nichols 튜닝

- 터미널 1: 로봇 시뮬레이션 런치 가동

```
cd ~/Workspace/ros2-ws  
source install/setup.bash  
ros2 launch my_robot_description pid_arm_launch.py
```

- Gazebo Classic 뷰어가 실행되며 회전 받침대 구조물 상단에 파란색 링크(Arm) 및 빨간색 끝단 질량(Payload)이 나타남
- 제어 입력이 아직 없으므로, 로봇 팔은 중력 작용 하에 바닥 방향으로 수직 처짐 상태가 됨

Step 1 — 제어 노드 구동 및 파라미터 초기화

- 비례 제어 단독 동작(P-only)을 위해 적분 및 미분 이득을 0으로 설정하여 노드 기동
- 터미널 2:

```
ros2 run my_robot_tools pid_arm_control.py \  
  --ros-args \  
  -p kp:=0.0 \  
  -p ki:=0.0 \  
  -p kd:=0.0 \  
  -p setpoint:=0.0
```

Step 2 — 임계 이득(K_u) 검출 탐색

- 비례 게인(K_p)을 영(0)에서부터 순차적으로 올리며 등진폭 진동 경계 지점 탐색

```
ros2 param set /pid_arm_controller kp 10.0
ros2 param set /pid_arm_controller kp 20.0
ros2 param set /pid_arm_controller kp 30.0
ros2 param set /pid_arm_controller kp 40.0
# ... 등진폭 오실레이션이 포착될 때까지 증가
```

관찰 거동 상태	상태 해석	후속 파라미터 조작 조치
진동 현상 없음	목표 위치 도달 지연 혹은 중력 오차 고착	K_p 이득 값 추가 상향 조정
감쇄형 진동	진동하며 점진적으로 위치 수렴	임계 한계 근접, K_p 미세 상향 조정
등진폭 지속 진동	수평선(0.0 rad) 주변 등진폭 진동 유지	임계 이득 K_u 포착 성공 및 기록
발산형 진동	오버슈트 증가로 진폭이 불규칙 발산	위험 상태, K_p 이득 즉시 감산 조정

Step 3 — 임계 주기(T_u) 데이터 계측

- 등진폭 진동이 유지되는 시점에서 파형의 주기를 직접 실측함

- 방법 A: 터미널 출력 데이터 타임스탬프 분석

- `/joint_states` 피드백 토픽 각도값의 최고점(또는 최저점)이 나타나는 시각 차이를 계산함

```
ros2 topic echo /joint_states
```

- ****방법 B: GUI rqt_plot 시각화 차트 계측 (권장)**** - 실시간 오차 및 각도 플롯 파형 그래프 상에서 마루와 마루 사이의 간격을 읽음 ``bash ros2 run rqt\_plot rqt\_plot /joint\_states/position[0] ``

Step 4 — 공식 대입을 통한 최종 게인 동조

- 측정한 임계값($K_u = 100$, $T_u = 0.5\text{s}$ 가정)을 Z-N 제2법칙 공식에 대입함

$$K_p = 0.6 \times 100 = 60$$

$$K_i = 1.2 \times \frac{100}{0.5} = 240$$

$$K_d = 0.075 \times 100 \times 0.5 = 3.75$$

- 계산 결과를 런타임 제어 노드에 동적 반영

```
ros2 param set /pid_arm_controller kp 60.0
ros2 param set /pid_arm_controller ki 240.0
ros2 param set /pid_arm_controller kd 3.75
```

- Z-N 튜닝 공식의 공격적인 게인 도출 특성

- Z-N 공식은 이전 진폭 대비 1/4 수준의 빠른 진폭 감쇄를 타겟으로 하므로, 매우 높고 과감한 게인을 계산해 냄
- 비선형성 물리 요소 및 디스크리트 제어 시스템 이산화 오차로 인해 계산된 $K_i = 240$ 을 가감 없이 대입할 시, 시뮬레이션상에서도 발산하거나 상하 요동이 크게 멈추지 않을 수 있음

- 필드 대응 및 완화법

- 안전 지향 튜닝을 위해 적분 이득(K_i)을 계산량의 1/5 ~ 1/10 수준(예: 30 ~ 50)으로 대폭 감산하여 안정도를 확보한 뒤 서서히 튜닝해 나감

Part 6. 외란 대처 및 미세 조정

페이로드 무게 가변에 따른 외란 대처 실습

- 제어 노드가 가동 중인 상태에서 하중을 2배(2.0 kg)로 증가시킨 후 반응 속도 관찰
- 터미널 1 (기존 런치 종료 후 재가동):

```
ros2 launch my_robot_description pid_arm_launch.py payload_mass:=2.0
```

- 터미널 2 (동작 명령 재인가):

```
ros2 run my_robot_description pid_arm_control.py \  
--ros-args -p kp:=60.0 -p ki:=50.0 -p kd:=3.75 -p setpoint:=0.0
```


발생 오작동 현상	유력한 원인	적정 매개변수 조율 가이드
각도 처짐 잔존 (목표 고도 유지 실패)	적분 제어량(K_i)의 누적력 부족 또는 지속 외력 증가	K_i 값을 소폭 인상하거나 피드포워드 <code>gravity_gain</code> 이득 상향
과도한 오버슈트 및 잦은 요동	비례 게인(K_p)이 과대하게 설정됨	K_p 값을 낮춤 (예: 60 ➡ 45~50 범위 조율)
고주파 미세 떨림 (채터링 발생)	미분 게인(K_d)이 눈금 노이즈를 과증폭시킴	K_d 값을 소폭 감산하거나 미분용 로우패스 필터 (LPF) 연산 설계
정착 도달 시간 지연 (느린 반응)	K_p 의 복원 복귀 거동 파워 혹은 K_d 의 오버슈트 억제력 결여	K_p 를 소폭 올리거나 감쇠 조율을 위해 K_d 를 미세하게 재설정

- 중력 보상 피드포워드 (gravity_gain) 적극 도입

- 수평 자세 제어 시에는 중력의 반대 방향으로 작용하는 최대 상쇄 토크가 수식상 요구됨

```
ros2 param set /pid_arm_controller gravity_gain 5.0
```

- 피드포워드로 중력 성분을 1차 상쇄해주면, PID 피드백 루프는 중력 부하를 크게 신경 쓸 필요 없이 오차 복원에만 자원을 집중할 수 있게 되어 훨씬 매끄러운 제어가 구현됨

- 오버슈트 배제 동조 (No-Overshoot Rule)

- 안전과 장비 보전이 최우선일 때, 계산된 Z-N 게인을 보수적으로 삭감 적용함

```
# Z-N 계산값의 약 절반 비율로 게인을 재조정된 안전 동조 예시
ros2 param set /pid_arm_controller kp 30.0
ros2 param set /pid_arm_controller ki 30.0
ros2 param set /pid_arm_controller kd 1.9
```

Part 7. 트러블슈팅



- ROS2 Control 구동 계통 활성화 여부 점검

```
# 로드된 조인트 컨트롤러의 액티브 상태 확인  
ros2 control list_controllers
```

- 제어 데이터 흐름 파이프라인 모니터링

```
# 계측 정보 및 제어 명령 토픽 발행 확인  
ros2 topic list | grep -E "joint_states|effort_controller"
```

- 데이터 실시간 갱신 주기 확인

```
# 조인트 수신 속도(Hz) 체크 (1000Hz 업데이트 타겟)  
ros2 topic hz /joint_states
```

- 활성화된 파라미터 세팅값 획득

```
# 특정 컨트롤러의 현재 이득 획득  
ros2 param get /pid_arm_controller kp
```